**WPI**

WORCESTER POLYTECHNIC INSTITUTE
ROBOTICS ENGINEERING PROGRAM

Lab 1 – System Architecture & Joint Control

SUBMITTED BY
Sukriti Kushwaha
Istan Slamet
Tracy Yang

Date Submitted: September 6, 2023

Date Completed: September 5, 2023

Course Instructor: Dr. Agheli

Lab Section: RBE 3001 A'12

Abstract

This report overviews the experiments performed in Lab 1, which was used to learn how to manipulate the movements of an OpenManipulator robotic arm. The experiments primarily included analysis of each joint in relation to one another while undergoing several different movements, with and without interpolation. The joint profiles were plotted against the movement times. All experiments were performed using MATLAB.

Introduction

The purpose of Lab 1 was to understand how to send inputs to the OpenManipulator arm to manipulate the movements of the individual joints while using GitHub to track our changes in MATLAB. We modified the Robot.m class to contain functions to:

- 1) Write joint values in degrees to the manipulator with and without interpolation,
- 2) Return the current set point joint positions, and
- 3) Return the end-of-motion joint set points.

Each function created had a corresponding issue and branch on GitHub, and our team members split up working on the functions to practice merging all the changes to the main branch. Creating these functions also allowed us to practice working with matrices and appending rows and vectors to capture the output data. We then wrote the output data to a .csv file and plotted the data using MATLAB to visualize the joint profiles. These actions comprised the first three signoffs of the lab.

For the fourth and final signoff, we sent the robot's base angle from 0 to 45 degrees with an interpolation time of a few seconds. While continuously reading the current joint positions, we also recorded the time stamps. With that data, we plotted the movements and found that the plot contained a linear slope due to application of constant force, resulting in constant velocity. The force applied to the joint is constant and results in a constant velocity and linear movement path.

For the lab report, we sent the robot's base angle to -90 degrees continuously three times first with and then without interpolation. From our plots, we observed that there were slight variations between the motions, which we attributed to be noise.

We also moved all four joints to varying positions with and without interpolation. We noticed that when the joint moved in the negative direction, its corresponding profile on the graph was mirrored horizontally. Additionally, without interpolation, the graph was steeper as the joints went directly to their given positions.

Methodology

Our team followed the instructions on the lab document very carefully. The first signoff required us to clone the repository and run MATLAB. Consulting the lab document, we completed the task. The second signoff asked us to make sure that we had our SSH keys set up and had finished Lab 0. Once done, we moved on to the next signoff, which required us to create several different functions for the motion of the arm. Each group member tackled 1 or 2 of the 5 functions, where on GitHub, issues and features were created for their respective function. After each function was created, we pull requested each branch into main, merging the completed functions on the same branch. We then created a separate script 'lab1.m' for testing. Once we completed signoff 3, where we ran the provided base code, we continued on to the fourth signoff, which required us to implement these functions, as well as loops and timers, to make the base joint rotate 45 degrees. The motion of the arm was plotted, and the time increments were recorded on a histogram.

For part 1 of the lab report experiment, we chose to move the robot from the origin to -90 degrees and back three times. To keep track of its joint positions and respective timestamps, we opted to store the values in three separate matrices for each iteration: three matrices for the three groups of timestamps and another three matrices for the three groups of joint readings. After obtaining these readings, we plotted them against each other to show the change in joint values over time.

For part 2 of the lab report experiment, we did the same as part 1, but instead of using `interpolate_js()` to move the arm, we used `servo_jp()`, which does not constrain the movement to a specific time.

For part 3 of the lab report experiment, we looped through the timestamps to find the incremental values between each, storing everything into a single matrix. From this single matrix, we were able to obtain mean, median, maximum, and minimum values.

For parts 4 and 6 of the lab report experiment, we moved each joint to distinct rotations with interpolation and then without interpolation. Each position was as follows: [30 0 0 0] [0 30 0 0] [0 0 30 0] [0 0 0 30]. Then we plotted each of the joints' individual profiles and compared the resulting profiles to visualize the difference between the interpolated and noninterpolated movements.

Results

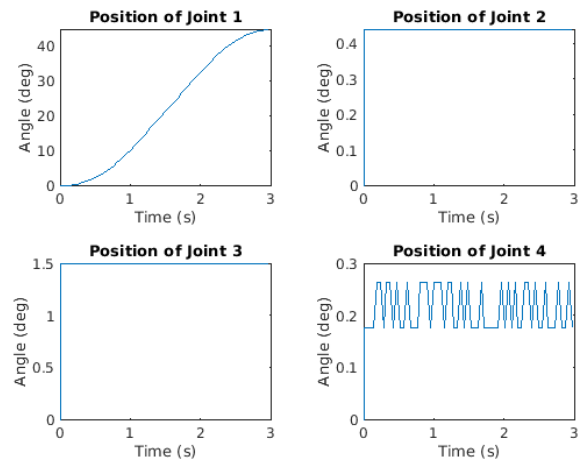


Figure 1: A depiction of the 4 joint's rotational position with respect to time.

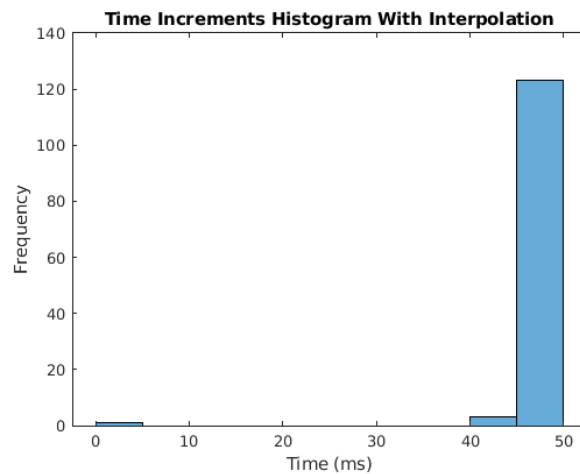


Figure 2: A histogram depicting the time incremental timesteps using the interpolate function.

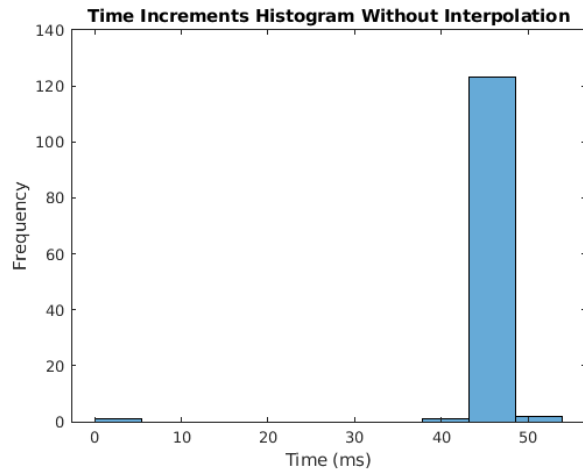


Figure 3: A histogram depicting the time incremental timesteps using the servo function.

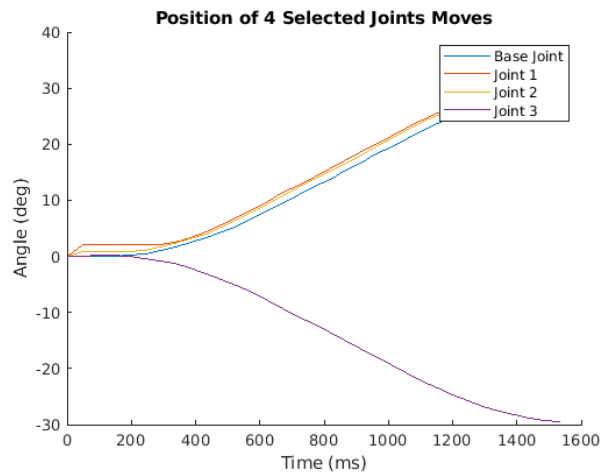


Figure 4: A plot graphing the rotational distance that the servos traveled in relation to time when interpolating.

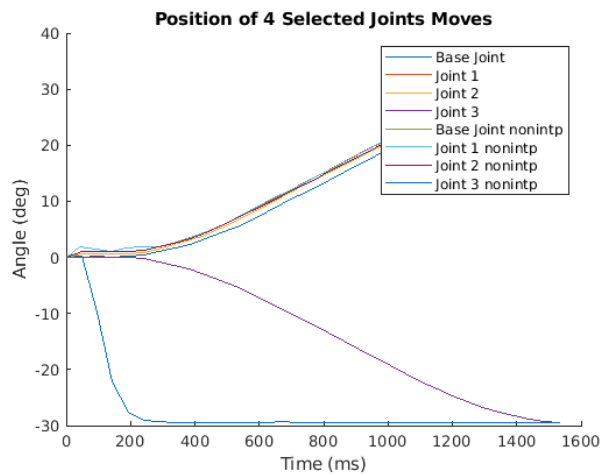


Figure 5: A plot overlaying figure 4 with the rotational distance in relation to time when noninterpolating.

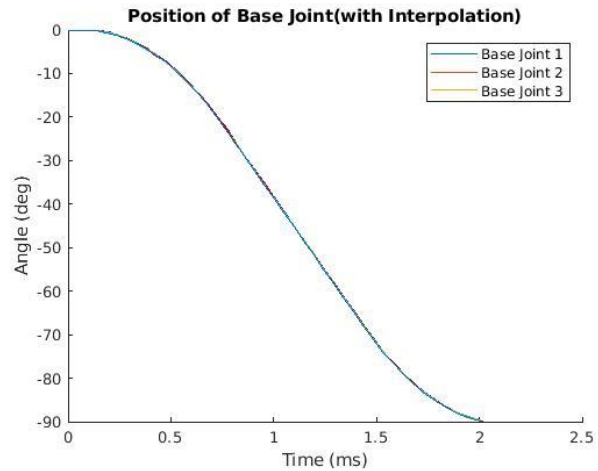


Figure 6: A plot showing 3 trials of the rotational distance of base joint in relation to time when using interpolation.

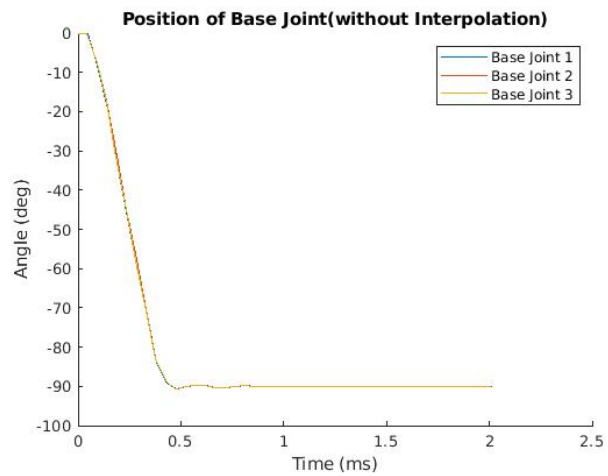


Figure 7: A plot showing 3 trials of the rotational distance of the base joint in relation to time without using interpolation.

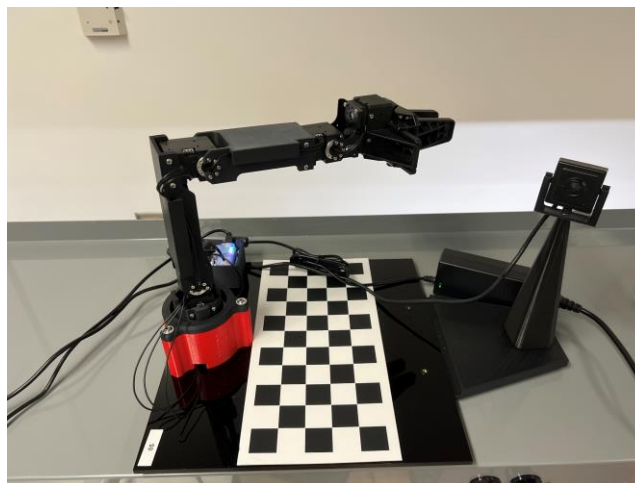


Figure 8: A picture of the robot in the first of four positions. The base joint is rotated to 30 degrees.

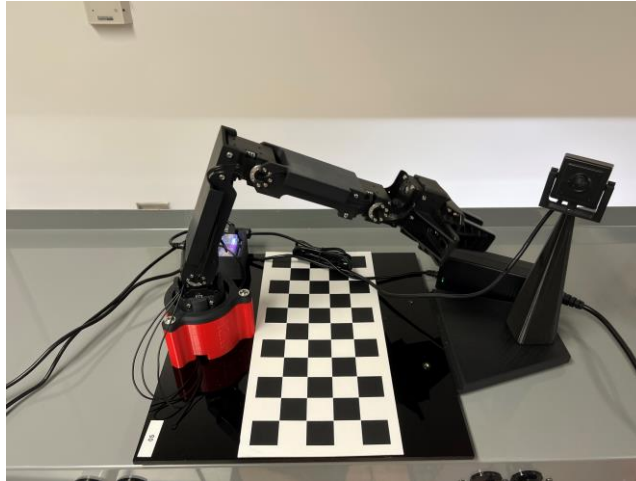


Figure 9: A picture of the robot in the second of four positions. The first joint is rotated to 30 degrees.

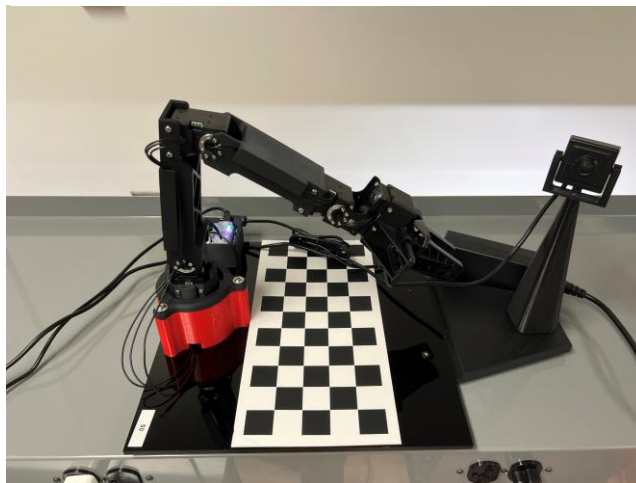


Figure 10: A picture of the robot in the third of four positions. The second joint is rotated to 30 degrees.

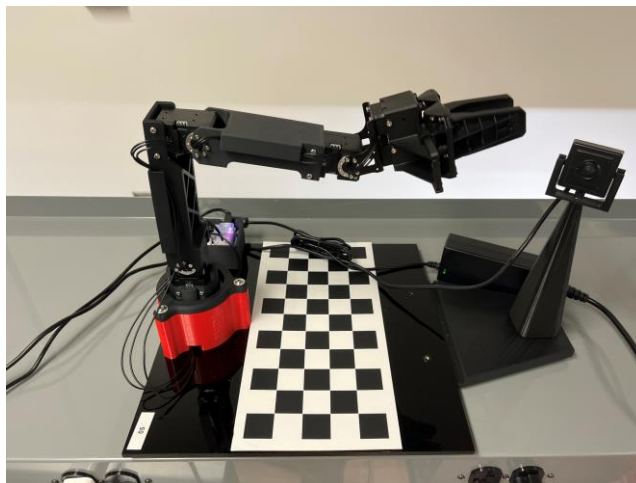


Figure 11: A picture of the robot in the last of four positions. The third joint is rotated to -30 degrees.

	With Interpolation (ms)	Without Interpolation (ms)
Mean	48.49	47.49
Median	47.99	47.99
Maximum	48.74	48.56
Minimum	0	0

Table 1: A quick summary of the mean, median, maximum, and minimum time step values for each trial.

Discussion

For signoff 4, we plotted the movements of each of the joints across a 45-degree rotation of the base joint, as seen in Figure 1. Expectedly, the base joint plot (in the top left subplot) created a positive slope that was linear. The force applied to the joint is constant and results in a constant velocity and linear movement path. As for the three other joints, it would be expected that their motion would be completely linear; however, some joints displayed slight movement. For example, joint 4 displayed a movement that oscillated up and down like a sine wave by about 0.1 degrees. However, joints 2 and 3 remained completely constant. We determined that because joint 4 is the farthest joint from the base and attached to the end-effector, the weight of the end-effector as well as the overall instability of the servo caused the inconsistencies.

For part 1 of the lab report, there were slight variations in the three sets of data, as seen in Figure 6. These changes can be attributed to noise and slight variations in the times between readings. Figure 2 displays the histogram that shows the differences in time increments, where most increments leaned towards 50ms.

For part 2 of the lab report, there were also slight variations in the three sets of data, as seen in Figure 7. These changes can be attributed to the fact that the use of `servo_jp()` ignores interpolation and the need for a target movement time. Thus, the arm moves rapidly from base position to goal position. The plot displays a steep curve, but the variations are more marginal than those found in question 1 because the rapid motion of the arm causes it to wobble. When the timing does not impact the motion, it is not completely necessary to use interpolate. Figure 3 shows the incremental timesteps for the movement without interpolation, and these values also lean towards 50ms.

For part 3 of the lab report, we found that there was a mean of about 50ms between each reading. This can be attributed to the fact that the MATLAB code we wrote could have been more efficient. The fact that we are also using the lab computers plays a part in the computation speed, whereas if we performed these experiments on stronger computers, the time between readings might be closer to 5ms. The mean of the interpolated movements was slightly higher than the non-interpolated movements, likely due to the time it took to move the joints for a set period of time.

For parts 4 and 6 of the lab report, we used the data (in our matrix) from our histogram, and from there, we used MATLAB functions to solve for the mean, median, maximum, and minimum time intervals, and these results can be found in Table 1. We did this for both functions `interpolate_js()` and `servo_js()`. We noticed that the joints sent to negative angles showed up as a horizontally mirrored profile on the graph. Additionally, when using interpolation, the profiles were smoother, as the joints took longer time to reach their set positions. Figures 8-11 show the poses that the arm actuated, and each pose uses a single servo to isolate the rotational changes, allowing us to better focus on the differences. One of the profiles on the

graph is noticeably steeper than the others in the negative direction. This is due to the fourth joint's attachment to the gripper without interpolation, as the weight of the gripper caused it to fall rapidly. The interpolated movement of this joint also had a negative slope, but it was not as steep.

Conclusion:

Overall, Lab 1 allowed us to get familiar with controlling the joint space of the OpenManipulator robot, with programming in the MATLAB environment, and with using Git to collaborate on our code. We created functions to help write joint values to the robot using interpolation and receive data such as current joint positions, velocities, and end-of-movement positions on individual Git branches. Each branch was merged into the main branch, where we used our functions to create MATLAB scripts to move the joints to specific angle values. We plotted the movement of the joints with respect to the time passed in between positions, which allowed us to learn how to store variables in vectors, manipulate matrices, and visualize them through linear graphs and histograms. Using the graphs, we were able to compare the joint profiles to each other and understand the effects of interpolation on joint movement.

Appendix A: Authorship

Section	Author
Introduction	Sukriti Kushwaha
Methodology	Istan Slamet
Results	Tracy Yang
Discussion	Everyone :)
Conclusion	Sukriti Kushwaha